A Comparative Study of Machine Learning Algorithms for Classifying LeetCode Problems by Difficulty Level

Luqman Muhammad Hardian
Applied Data Science Study Program, Department of Informatics
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
luqmanhardian@gmail.com

***Abstract*—Online coding platforms provide thousands of practice problems, but the difficulty labels of these problems are usually assigned manually. Manual labeling is time-consuming and can be subjective. This study builds a text classification model that predicts the difficulty level of LeetCode problems (Easy, Medium, and Hard) automatically. The data were collected from the official LeetCode platform by using its GraphQL interface and a web scraping library, producing 4,047 raw records that were reduced to 3,096 usable samples. The text went through cleaning, case folding, tokenization, and stopword removal. Each problem was then represented as a 300-dimensional document vector using pretrained fastText embeddings. Three algorithms were compared: Logistic Regression, Random Forest, and Support Vector Machine (SVM). Each model was tuned with grid search and stratified five-fold cross validation, and the models were evaluated on a held-out test set using macro-averaged F1-score as the main metric. The results show that SVM achieved the best macro-F1 of 0.452 and the best balance across classes, while Random Forest obtained the highest accuracy of 0.539 but a low macro-F1 of 0.370 because it ignored the minority classes. The Hard class was the hardest to classify, with the lowest F1-score of 0.375, because its descriptions overlap with those of Medium problems, which in turn lie between the Easy and Hard extremes. The study shows that a simple and efficient pipeline based on pretrained embeddings and a margin-based classifier can be a practical baseline for automatic difficulty classification.**

***Index Terms*—text classification, LeetCode, fastText, support vector machine, Random Forest, difficulty classification**

I. Introduction

Programming practice platforms such as LeetCode host a large number of coding problems that are grouped by difficulty level and by topic. Grouping problems by difficulty is useful for several reasons. It helps learners follow a gradual learning path, supports interview preparation, and allows recommender systems to suggest problems that match the current skill level of a user [1].

However, the difficulty label of a problem is usually assigned manually by the platform. Manual labeling takes a lot of time when the number of problems grows, and it can be subjective because two people may judge the same problem differently. An automatic approach is therefore needed to assign difficulty labels in a consistent and scalable way.

Difficulty can be predicted from the textual content of a problem, such as its title and its description [2]. This makes the problem suitable for text mining. Text classification techniques can learn patterns from the problem text and map them to a difficulty class. With a machine learning model, new problems can be labeled automatically.

This study has three objectives. First, it builds a dataset of LeetCode problems by crawling the platform. Second, it compares three machine learning algorithms for the difficulty classification task. Third, it analyzes which class is the most difficult to classify and explains the reasons. The main contributions are a crawled dataset of 3,096 problems, a reproducible comparison of three algorithms with hyperparameter tuning, and an analysis of the per-class behavior of each model.

The rest of this paper is organized as follows. Section II reviews related work. Section III explains the methodology, including data collection, preprocessing, feature extraction, and the experiment design. Section IV presents and discusses the results. Section V gives the conclusion.

II. Related Work

Text classification is one of the most fundamental tasks in natural language processing. Li et al. [3] reviewed text classification approaches published between 1961 and 2021. Their survey covers models from traditional machine learning to deep learning, and it compares the way each family extracts features and classifies text, together with the benchmark datasets and the evaluation metrics that are commonly used. This survey gives the background for the classical classifiers and the metrics used in the present study.

Several studies have applied text classification to programming problems. Iacob et al. [4] built AlgoLabel, a dataset of algorithmic challenges collected mainly from Codeforces, and defined a multi-label task that predicts the type of algorithmic solution from the problem statement, the solution code, or both. For the text-only task they compared Random Forest, SVM, and XGBoost on TF-IDF features, a fine-tuned BERT model, and their own recurrent neural network. SVM and Random Forest reached micro-averaged F1-scores of 0.55 and 0.54, XGBoost reached 0.60, the fine-tuned BERT model

reached only 0.40, and their proposed network reached 0.62. They relate the weak result of BERT to the small size of the dataset.

De Souza et al. [5] classified Codeforces problems into ten topic labels, again as a multi-label task. They compared LSTM and BiLSTM with Word2Vec embeddings, BERT, and Llama 3.1-8B, using the problem statement alone and the statement together with source code. The macro-F1 of LSTM, BiLSTM, and BERT stayed between 0.33 and 0.36, while Llama 3.1-8B reached 0.79 on the statements alone, although with a large standard deviation across folds. Adding the source code did not improve the results.

Other studies are closer to the difficulty prediction task of this paper. Skarbalius and Lukosevičius [2] studied the automatic evaluation of the difficulty of programming problems. In their dataset each problem consists of a text description and figures, and they evaluated difficulty from the text, from the images, and from a combination of both, reporting first results together with a discussion of future work. Zhou and Tao [6] proposed Multi-task BERT, a BERT-based model that shares representations between tasks, to predict the difficulty of programming problems as an alternative to manual evaluation.

Compared with the studies above, the present work does not propose a new algorithm. Instead, it applies existing methods to a specific and practical domain, namely the difficulty classification of programming problems. In contrast to [4] and [5], which predict algorithm or topic tags, this study predicts the difficulty label. In contrast to [2], which also analyzes figures, and to [6], which fine-tunes a BERT-based model, this study uses only the problem text, averaged pretrained word embeddings, and three classical classifiers. The dataset is collected directly from the official LeetCode platform through its GraphQL interface, and the macro-averaged F1-score is emphasized to expose the effect of class imbalance.

III. Methodology

Fig. 1 shows the overall research pipeline, from data collection to evaluation.

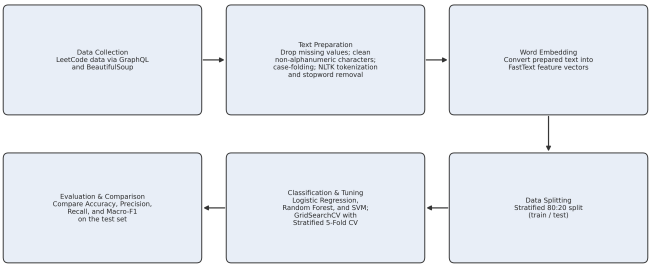


Fig. 1. Research pipeline.

A. Data Collection

The data were collected from the official LeetCode platform [7] on October 6, 2026. The crawling process used two components. The first was the official GraphQL interface, which returns structured problem data such as the problem identifier, the title, the difficulty label, and the topic tags. The second was the Beautiful Soup library, which was used to parse parts of the problem page that are delivered as HTML. The attributes that were taken for each problem are the problem identifier, the title, the problem description, and the difficulty label. The raw crawl produced 4,047 records.

B. Dataset Description

After the raw data were collected, records with a missing problem description were removed, because a missing description usually means that the problem is a premium problem whose content is not publicly available. The topic and category columns were also dropped during preparation, so the final dataset only uses the difficulty label as the target. The final dataset contains 3,096 problems with three classes: Easy, Medium, and Hard. Table I shows the distribution of the classes. The dataset is imbalanced, because the Medium class has more than twice the number of samples of the Easy and Hard classes. This imbalance becomes an important factor in the discussion of the results. Fig. 2 shows the class distribution.

TABLE I
Distribution of Difficulty Classes

Class	Number of Problems	Proportion (%)
Easy	754	24.35
Medium	1603	51.78
Hard	739	23.87
Total	**3096**	**100.00**

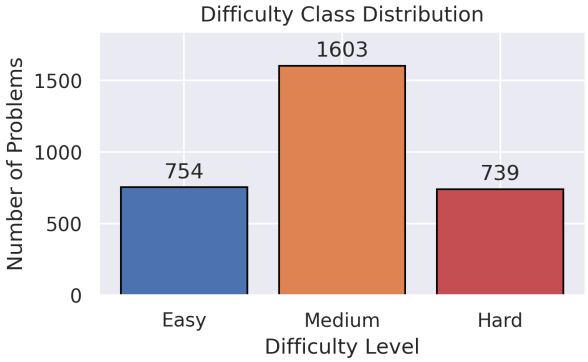


Fig. 2. Distribution of problems per difficulty class.

C. Preprocessing

Each problem text went through several preprocessing steps. First, records with a missing description were removed. Second, text cleaning removed non-alphanumeric characters that were left over from HTML markup and other special characters. Third, case folding converted all letters to lowercase. Fourth, tokenization split the text into word tokens using the `word_tokenize` function from NLTK [8]. Fifth, stopword removal deleted common English stopwords using the stopword corpus provided by NLTK. Table II shows an example of the text before and after preprocessing.

TABLE II
EXAMPLE OF TEXT BEFORE AND AFTER PREPROCESSING

Before	After
"Given an array of integers nums and an integer target, return indices..."	*"given array integers nums integer target return indices"*

D. Feature Extraction

Text representation was done with pretrained fastText word embeddings [9]. Each token was mapped to a 300-dimensional word vector, and all word vectors in a document were averaged into a single 300-dimensional document vector. This document vector was then used as the input feature for the classifiers. Formally, for a document with n tokens $w_1, \ldots, w_n$ and a word vector function $v(\cdot)$, the document vector d is computed as:

$$d = \frac{1}{n} \sum_{i=1}^{n} v(w_i). \tag{1}$$

E. Classification Algorithms

Three algorithms were compared. They are classical models of the kind reviewed in [3]. Logistic Regression is a linear model that predicts class probabilities by applying a logistic function to a weighted sum of the input features. It is simple, fast, and easy to interpret. Random Forest is an ensemble of decision trees; each tree is trained on a random subset of the data and features, and the final prediction is decided by majority voting. It can capture non-linear relations but tends to be biased toward the majority class when the data are imbalanced. Support Vector Machine is a margin-based classifier that searches for a decision boundary with the largest margin between classes; with an RBF kernel it can model non-linear boundaries.

The three models were chosen to represent different learning principles: a linear probabilistic model, an ensemble of trees, and a margin-based model. This selection also differs from the model sets used by other members of the group. The aim here is to compare a linear approach with a non-linear ensemble and a kernel-based method, so that the effect of the model principle on the balanced performance can be observed clearly.

Each model was tuned with grid search. The parameter grids are shown in Table III. The class weight was set to *balanced* to reduce the effect of class imbalance.

TABLE III
HYPERPARAMETER GRIDS

Model	Parameters
Logistic Regression	$C \in \{0.1, 1, 10\}$
Random Forest	*n_estimators* $\in \{200, 300, 500\}$, *max_depth* $\in$ {None, 20, 40}
SVM	$C \in \{0.1, 1, 10\}$, kernel $\in$ {linear, rbf}, gamma $\in$ {scale, auto}

F. Experiment and Evaluation

The dataset was divided into a training set (80%) and a test set (20%) using stratified sampling, so that the class proportions were preserved in both sets. Hyperparameter tuning used grid search with stratified five-fold cross validation on the training set only. The best configuration was then retrained on the full training set and evaluated on the test set.

The main metric is the macro-averaged F1-score. Macro-F1 gives equal weight to every class, so a model that ignores the minority classes receives a low score. Accuracy is reported as well, but it is less reliable when the data are imbalanced. Precision, recall, and weighted F1 are also reported for completeness. The experiment was done in Python using scikit-learn [10], NLTK [8], and fastText [9].

IV. RESULTS AND DISCUSSION

A. Classification Results

Table IV shows the performance of all three models on the test set. The SVM achieved the best macro-F1 of 0.452, followed by Logistic Regression with 0.432 and Random Forest with 0.370. Random Forest obtained the highest accuracy of 0.539, but its macro-F1 was the lowest. Fig. 3 compares the four metrics for each model.

TABLE IV
MODEL PERFORMANCE ON THE TEST SET

Model	Acc.	Prec.	Rec.	F1-m	F1-w
SVM	0.455	0.456	0.482	**0.452**	0.457
Logistic R.	0.437	0.434	0.454	0.432	0.442
Random F.	**0.539**	0.530	0.395	0.370	0.455

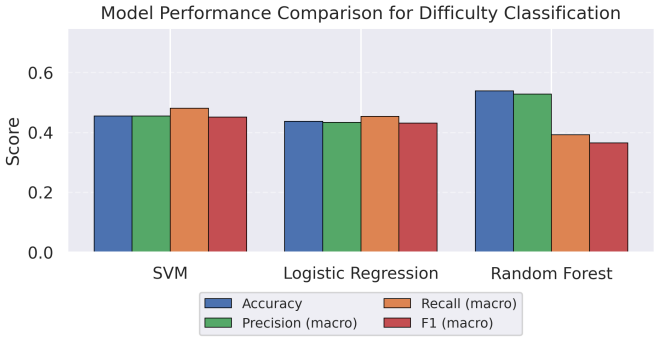


Fig. 3. Comparison of model performance on difficulty classification.

B. Per-Class Analysis

Table V shows the per-class results of the best model, the SVM. The Easy class obtained the highest F1 of 0.510, while the Hard class obtained the lowest F1 of 0.375. The Medium class had a high precision of 0.609 but a low recall of 0.383. Fig. 4 shows the confusion matrix of the SVM, and Fig. 5 shows the per-class metrics.

TABLE V
PER-CLASS RESULTS OF THE SVM MODEL

Class	Precision	Recall	F1	Support
Easy	0.435	0.616	0.510	151
Medium	0.609	0.383	0.470	321
Hard	0.324	0.446	0.375	148

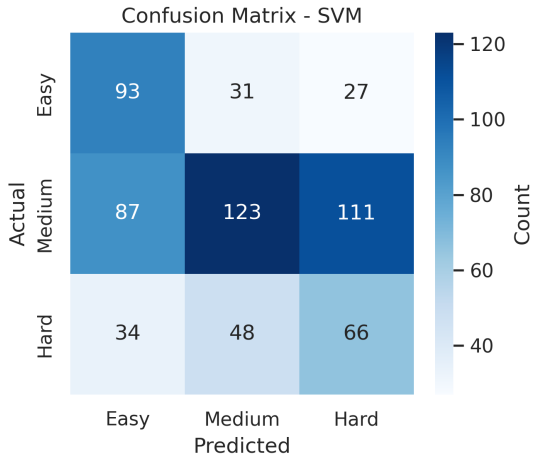


Fig. 4. Confusion matrix of the SVM model.

C. Discussion

The SVM gave the best balance across the three classes. Because it maximizes the margin and uses a balanced class weight, it does not collapse to the majority class. Logistic Regression behaved similarly, since it is also a linear model with a balanced class weight, but its macro-F1 was slightly lower.

Random Forest showed a different pattern. Its accuracy was the highest, but its macro-F1 was the lowest. This happens because the data are imbalanced: the tree ensemble learned to predict the Medium class very often, which raises the accuracy but hurts the minority classes. The recall of the Easy class dropped to 0.179 and the recall of the Hard class dropped to 0.095. This confirms that accuracy alone can be misleading for imbalanced text classification.



Fig. 5. Per-class precision, recall, and F1 of the SVM model.

The Hard class was the hardest to classify, since it obtained the lowest precision (0.324) and the lowest F1-score (0.375) of the three classes. The Medium class had the lowest recall (0.383): from the confusion matrix, many Medium problems were predicted as Easy or Hard. This is reasonable, because Medium problems lie between the Easy and Hard extremes, so their text often contains features that belong to both neighboring classes. As a consequence, many problems that are predicted as Hard do not actually belong to the Hard class, which lowers its precision, and the descriptions of Hard problems overlap with those of Medium problems. The class size alone does not explain this difficulty, because the Hard class (739 problems) and the Easy class (754 problems) have almost the same number of samples, yet the Easy class obtained the highest F1-score of 0.510.

The hyperparameter tuning also gave useful insight, as shown in Fig. 6. For the SVM, the RBF kernel with $C = 1$ and gamma set to *scale* gave the best cross-validation score. For Logistic Regression, a small value of $C = 0.1$ was best, which suggests that stronger regularization helps the model generalize. For Random Forest, a limited depth of 20 worked better than unlimited depth, which indicates that very deep trees overfit the data.

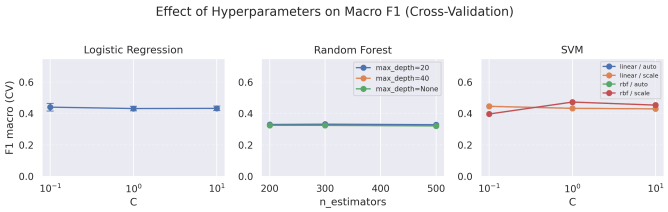


Fig. 6. Effect of hyperparameters on cross-validated macro-F1.

This study has several limitations. The dataset contains only 3,096 problems, and the classes are imbalanced. The difficulty labels come from the platform and may be subjective. The features use averaged word embeddings, which ignore the order of words and the structure of the problem. Feature extraction does not include the code or the constraints of each problem, which may carry useful information about difficulty. Compared with the related work in Section II, the macro-F1 of 0.452 obtained here is higher than the macro-F1 of 0.33 to 0.36 that de Souza et al. [5] reported for LSTM, BiLSTM, and BERT, but their task is topic tagging, so this comparison is only indicative. The classical classifiers of this study also behave like those in [4], where SVM and Random Forest were close to each other and a fine-tuned BERT model did not outperform them on a small dataset. At the same time, the low absolute F1 values show that difficulty classification from the problem text alone remains a difficult task.

V. CONCLUSION

This study compared three machine learning algorithms for classifying LeetCode problems by difficulty level. The data were collected from the official platform, preprocessed with

standard text processing steps, and represented with pretrained fastText embeddings. The SVM achieved the best macro-F1 of 0.452 and the most balanced performance across classes. Random Forest obtained the highest accuracy but the lowest macro-F1 because it ignored the minority classes. The Hard class was the hardest to classify, with the lowest F1-score of 0.375, because its descriptions overlap with those of Medium problems, which lie between the Easy and Hard extremes. For this task, the SVM with an RBF kernel and a balanced class weight is the recommended model.

Future work can extend this study in several directions. More data can be collected to reduce class imbalance. Transformer-based models such as BERT can be tested to capture context better. Multi-label classification can be used to combine difficulty and topic labels. Finally, features from the problem code and constraints can be added to enrich the representation.

ACKNOWLEDGMENT

The author would like to thank Dr. Tita Karlita, S.Kom., M.Kom., the lecturer of the Text Mining course, for her guidance. The author also thanks fellow team member Trio Setiawan for sharing the dataset and notebook, from the initial stages of the project through the word embedding stage. Sonnet 5.5 was used to assist in preparing this paper, particularly with the IEEE writing style and English-language writing.

REFERENCES

[1] B. Vesin, K. Mangaroska, K. Akhuseyinoglu, and M. Giannakos, "Adaptive assessment and content recommendation in online programming courses: On the use of Elo-rating," *ACM Transactions on Computing Education*, vol. 22, no. 3, Art. no. 33, pp. 1–27, 2022, doi: 10.1145/3511886.

[2] A. Skarbalius and M. Lukoševičius, "Automatic programming problem difficulty evaluation – First results," in *Information and Software Technologies (ICIST 2021)*, Communications in Computer and Information Science, vol. 1486. Cham, Switzerland: Springer, 2021, doi: 10.1007/978-3-030-88304-1_12.

[3] Q. Li *et al.*, "A survey on text classification: From traditional to deep learning," *ACM Transactions on Intelligent Systems and Technology*, vol. 13, no. 2, Art. no. 31, 2022, doi: 10.1145/3495162.

[4] R. C. A. Iacob, V. C. Monea, D. Rădulescu, A.-F. Cepă, T. Rebedea, and Ş. Trăuşan-Matu, "AlgoLabel: A large dataset for multi-label classification of algorithmic challenges," *Mathematics*, vol. 8, no. 11, Art. no. 1995, 2020, doi: 10.3390/math8111995.

[5] B. V. de Souza, A. S. S. Silvestre, V. H. F. Lisboa, and V. R. P. Borges, "Pre-trained language models for multi-label text classification of competitive programming problems," in *Anais do XXI Encontro Nacional de Inteligência Artificial e Computacional (ENIAC)*, Belém, Brazil, 2024, pp. 73–84, doi: 10.5753/eniac.2024.245222.

[6] Y. Zhou and C. Tao, "Multi-task BERT for problem difficulty prediction," in *Proc. Int. Conf. Communications, Information System and Computer Engineering (CISCE)*, 2020, pp. 213–216.

[7] LeetCode, "LeetCode online programming platform," 2026. [Online]. Available: https://leetcode.com

[8] NLTK Project, "NLTK documentation: Natural language toolkit," 2026. [Online]. Available: https://www.nltk.org

[9] Facebook Research, "fastText documentation: Library for efficient text classification and representation learning," 2026. [Online]. Available: https://fasttext.cc

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.